

Probing the Capability Ceiling of an LLM Coding Agent for Volta-Specialized GPU Kernel Optimization

Final Project — Parallel Programming, Spring 2026

Team 37

National Taiwan University

Parallel and Distributed Programming (114-2)

Evaluated on the NCHC Taiwan 2 NVIDIA Tesla V100 cluster

<https://github.com/Distant22/KernelBench>

Abstract—We study how far a general-purpose large-language-model (LLM) coding agent can push GPU-kernel performance when it is locked to a deliberately “old” target: the NVIDIA Tesla V100 (Volta, CC 7.0), in FP32, with Ampere/Hopper features (TF32, native BF16, `cp.async`, WGMMMA) forbidden. Over a fixed 30-problem KernelBench subset (15 single operators, 10 fused operators, 5 end-to-end models) we drive GitHub Copilot Chat through a strict four-step chain-of-thought protocol and a mandatory Nsight-guided measure-attribute-revise loop, then evaluate every `ModelNew` on a real V100 against both PyTorch eager and `torch.compile` (Inductor) baselines. All 30/30 solutions are numerically correct; the agent reaches a geometric-mean speedup of $0.92\times$ over eager and $0.89\times$ over compile, with 19/30 problems at or above eager parity and headline wins up to $2.2\times$ (depthwise convolution), $1.9\times$ (masked cumsum), and $1.6\times$ (a Mamba-2 layer). Our central finding is a sharp, architecture-induced capability ceiling: the agent reliably wins on memory-bound operators by fusing passes and cutting HBM traffic, but cannot beat cuBLAS/cuDNN on compute-bound paths already at 90–95% of FP32 peak. A profile-driven root-cause analysis explains every loss.

Index Terms—GPU kernels, LLM coding agents, KernelBench, Triton, V100, roofline analysis

Project Category: ☐ A (Self-chosen topic) ☒ B (AI-agent code optimization)
Tick exactly one. This tells reviewers which lens to apply.

Code and data: all solutions, the evaluation harness, the Nsight profiling driver, and the raw measurement logs are available at <https://github.com/Distant22/KernelBench>.

I. INTRODUCTION

Hand-writing high-performance GPU kernels couples numerical-algorithm design with a detailed mental model of a specific architecture’s memory hierarchy, occupancy, and instruction-level parallelism. Modern LLM coding agents can already emit syntactically valid CUDA and Triton; the open question is whether they can emit *fast* kernels, and where their performance ceiling lies relative to highly tuned vendor libraries.

This paper (Category B: AI-agent analysis and optimization of parallel programs) attacks that question under an intentionally adversarial constraint. Instead of the newest GPU, we pin the target to the **NVIDIA Tesla V100-SXM2-32GB** (Volta, CC 7.0) and to **FP32**, forbidding every post-Volta acceleration path: TF32, native BF16 tensor cores, asynchronous copies (`cp.async`), and Hopper WGMMMA. On V100 FP32 the vendor libraries (cuBLAS, cuDNN) are mature and already near the hardware roofline, so any speedup the agent earns must come from genuine algorithmic or memory-traffic improvements rather than from opportunistically dropping to a lower-precision tensor-core path. The constraint turns the benchmark into a clean probe of the agent’s *reasoning* about the memory hierarchy.

Contributions. (i) A reproducible pipeline that drives an LLM coding agent through a four-step chain-of-thought (CoT) protocol over a *fixed* 30-problem KernelBench subset and closes the loop with an Nsight Systems/-Compute profiling driver that attributes each bottleneck on the V100. (ii) A full V100 FP32 measurement: 30/30 correct, geometric-mean $0.92\times$ over eager, with the complete per-problem table and `fast_p` curve. (iii) An architecture-grounded, profile-backed account of *where the agent wins and loses*—a memory-bound “win zone” versus a cuBLAS/cuDNN-dominated “ceiling zone.” (iv) Documentation of four evaluation-harness pitfalls and the agent-side fixes required for a fair, leak-free comparison.

II. BACKGROUND AND RELATED WORK

A. KernelBench and the `fast_p` metric

KernelBench [1] benchmarks LLM-generated GPU kernels. Each problem ships a PyTorch reference `Model`; a candidate must provide a drop-in `ModelNew` with an identical interface. The harness checks numerical correctness with `torch.allclose` over randomized inputs, then measures wall-clock runtime. The recommended aggregate

metric is `fast_p`: the fraction of problems that are simultaneously correct *and* achieve speedup $\geq p$. We report `fast_p` against both the eager and the `torch.compile` (Inductor) baseline.

Dataset and subset. We fix 30 problems in advance (file `tasks.txt`) to remove any cherry-picking degree of freedom: 15 Level-1 single operators (GEMM variants, softmax, RMSNorm, reductions, several convolution types, masked cumsum, scaled dot-product attention), 10 Level-2 fused operators (e.g. Conv+ReLU+Bias, Gemm+activation+reduction chains, Conv+Scale+Sigmoid+GroupNorm), and 5 Level-3 end-to-end models (MLP, Vision Transformer, minGPT causal attention, a minGPT block, a Mamba-2 SSD layer). This spans the difficulty gradient from a single kernel to a full model.

B. The Volta (V100) target

The V100 provides 80 SMs, up to 96 KB shared memory per SM, and ~ 900 GB/s HBM2 [4]. Its tensor cores accelerate FP16 matmul, but it has *no* TF32/native-BF16 path and no asynchronous global-to-shared copy. For FP32 the practical envelope is set by the ~ 15.7 TFLOP/s FP32 peak (compute-bound) and the ~ 900 GB/s bandwidth (memory-bound). The roofline ridge point is $\approx 15,700/900 \approx 17$ FLOP/byte: most elementwise, normalization, reduction and attention-epilogue operators sit far to its left and are bandwidth-limited, while dense GEMM and convolution sit to its right and are compute-limited—precisely where cuBLAS/cuDNN are hardest to beat.

C. Triton and torch.compile

Triton [2] is a Python-embedded DSL that compiles tile-level programs to GPU code, exposing block sizes, shared-memory staging, and software pipelining. `torch.compile` with the Inductor backend automatically generates Triton for many subgraphs and is a strong, fully automatic baseline; beating it is harder and more meaningful than beating eager, so we report both. FlashAttention [3] motivates the fused-attention path we use on Level-3 attention problems.

III. METHODOLOGY

A. The four-step CoT agent protocol

The agent (GitHub Copilot Chat) is governed by a role/system prompt (Appendix B) that casts it as an HPC specialist and *forbids emitting code before analysis*. For each problem it must output, in order: (1) **operator characterization**—the math, dtype/shape, and a compute-vs. memory-bound classification with an arithmetic-intensity argument; (2) **memory and tiling strategy**—V100 block/grid tiling, shared-memory and register budget, and the coalescing pattern; (3) **hardware-conflict reduction**—bank-conflict and divergence avoidance, ILP/occupancy plan; (4) **implementation**—the Triton or CUDA `ModelNew`.

B. Nsight-guided profile-driven loop

A single wall-clock number tells the agent only *that* a kernel is slow, not *why*. Every correct-but-slow solution is routed through a profiling driver (`profile_feedback.py`) submitted to a V100 compute node; it refuses to run off a compute host, since profiling is forbidden on the login node. Evidence is collected in two layers matching the two questions an optimizer must answer:

- **Nsight Systems (nsys)**—“**where does time go?**” The full forward timeline for candidate, eager, and compile paths under an identical seed/input, reporting per-kernel duration, kernel count, and CUDA-API/launch overhead.
- **Nsight Compute (ncu, --deep)**—“**why is *that* kernel slow?**” For only the kernels covering $\geq 80\%$ of GPU time (capped at three per path) it collects SpeedOfLight, LaunchStats, Occupancy, MemoryWorkloadAnalysis, and SchedulerStats sections, yielding per-kernel SM-/memory-throughput, occupancy, eligible-warps, and register/shared-memory figures.

The driver labels the dominant bottleneck (compute-/memory-/occupancy-/launch-bound or library-dominated) using fixed thresholds. This feeds a *mandatory* loop the prompt enforces for every change: (1) measure the three profiles; (2) attribute cost to a specific kernel/bottleneck using counters, not intuition; (3) propose exactly *one* scoped change with a predicted effect; (4) back up the current file; (5) re-profile and compare before/after; (6) keep only if faster *and* still correct, else revert and record why. An explicit *exhaust-all-directions* policy requires checking data-reuse hoisting, kernel fusion, constant-folding into weights, `torch.compile` dispatch, launch-overhead reduction, and a systematic tiling sweep before any problem is declared at its ceiling. An *honest-attempt* policy forbids silently substituting a vendor library to flatter the numbers: a sub-1.0 $\times$ hand-written result is reported as-is.

IV. EXPERIMENTAL SETUP

All measurements use one NVIDIA Tesla V100-SXM2-32GB on the NCHC Taiwania 2 cluster, with PyTorch 2.5.1 (cu121), Triton 3.1.0, and CUDA runtime 12.1; profiling uses Nsight Systems 2024.6.1 and Nsight Compute 2024.3.2. The GPU architecture flag is `["Volta"]` and the working precision is FP32 throughout. Each solution is evaluated in an *isolated subprocess* by `run_eval_all.py`: it loads the reference, runs the correctness check (5 randomized trials, `torch.allclose`), times the custom kernel (100 trials, CUDA events), then times the eager and `torch.compile` references (100 trials each) on the same device. Subprocess isolation is essential because several solutions install module-level monkey-patches whose effects must not leak across problems. Speedup is $t_{\text{baseline}}/t_{\text{kernel}}$. We define **success** as “correct

TABLE I
AGGREGATE V100 FP32 RESULTS OVER THE FIXED 30-PROBLEM
SET. **FAST_P** = FRACTION CORRECT AND SPEEDUP $\geq p$.

Metric	Value			
Compiled	30/30			
Correct	30/30			
Geo-mean speedup (vs. eager)	0.92×			
Geo-mean speedup (vs. compile)	0.89×			
fast_p	$p=1.0$	$p=1.5$	$p=2.0$	$p=3.0$
vs. eager	19/30	4/30	0/30	0/30
vs. compile	13/30	2/30	1/30	0/30

and speedup ≥ 1.0 ” (**fast_1**) and report it against both baselines; all runs are launched via SLURM (**sbatch run_eval_all.sbatch**).

A. Evaluation-harness pitfalls and agent-side fixes

A fair comparison required diagnosing four harness behaviors. **(P1) Static checker rejects valid Triton:** the optional anti-cheat checker only recognizes a `CUDA __global__` entry point; we disable it (`check_kernel=False`)—the correctness check is unaffected. **(P2) Source introspection fails for @triton.jit:** the default loader `exec()`s the candidate, after which Triton cannot recover kernel source (`OSError`); we use the temp-file loader (`backend=triton`). **(P3) OOM in allclose:** for the 6.4 GB softmax problem the C++ `isclose` peaks near 38 GB and OOMs the 32 GB V100; affected solutions install a chunked, numerically identical streaming `allclose`. **(P4) Dropout RNG mismatch:** the harness does not re-seed between the reference and candidate forwards, so dropout masks diverge and even a verbatim copy fails; we neutralize dropout to the identity (a no-op at inference).

V. RESULTS

Table I reports the aggregate metrics and the **fast_p** curve; Table II gives the complete per-problem breakdown. All 30/30 solutions are numerically correct.

Level 1 (single operators). The results cleanly separate two regimes. *Memory-bound* operators are the agent’s strongest territory: a two-pass streaming RMSNorm and an online streaming softmax beat eager by $1.7\times$ and $1.4\times$, a fused mask+scan masked cumsum reaches $1.9\times$, and a direct 3×3 depthwise convolution beats even cuDNN (up to $2.2\times$ under `torch.compile`). In contrast, every *compute-bound* GEMM variant lands at $0.7\text{--}0.9\times$: cuBLAS FP32 already runs at 90–95% of peak while a portable Triton tile reaches only $\sim 55\text{--}65\%$. The same logic governs the five convolution/attention problems that cuDNN or the fused-SDPA path dominates (P50/P56/P61/P76/P97); all are correct but lose, from $0.64\times$ (dilated Conv1D) and $0.51\times$ (large-stride Conv2D) down to $0.15\times$ (flash-attention).

Level 2 (operator fusion). The standout is Conv+Bias+Scale+Sigmoid+GroupNorm ($1.45\times$ eager):

the agent collapses five elementwise/reduction passes into two, eliminating three HBM round-trips over the activation tensor. Two further wins come from *algebraic simplification*: $y \cdot s + y \equiv y \cdot (s+1)$, and a clamp/Log-SumExp/Mish chain reduced to a single post-pass. The remaining GEMM-driven chains are bounded near $1.0\times$ because cuBLAS dominates their runtime.

Level 3 (end-to-end models). The largest wins come from causal attention: replacing a manual $QK^\top \rightarrow \text{mask} \rightarrow \text{softmax} \rightarrow V$ with `F.scaled_dot_product_attention(is_causal=True)` routes to V100’s memory-efficient fused-attention path and yields $1.5\times$ (minGPT attention) and $1.4\times$ (minGPT block), beating `torch.compile` too. The MLP reaches $1.07\times$ eager via a `torch.compile` wrap; the small ViT ($B=2$) sits at $0.99\times$ compile, within noise; and the Mamba-2 SSD layer—despite being numerically delicate, since its `exp(cumsum(.))` forbids reordering its contraction—is pushed to $1.58\times$ eager by hoisting its parameter-only contraction products into a cache computed once.

VI. DISCUSSION AND ANALYSIS

A single axis—arithmetic intensity relative to the V100 roofline ridge—predicts the agent’s success better than problem “difficulty” or level.

The win zone (memory-bound). When a kernel is bandwidth-limited, the optimal move is to minimize HBM traffic, exactly what the four-step CoT elicits: count passes, fuse them, keep intermediates in registers/SRAM, stream rows that exceed shared memory. Softmax, RMSNorm, depthwise convolution, fused GroupNorm epilogues, and fused attention all fall here, earning $1.1\text{--}2.2\times$.

Root-cause analysis of the failures (ceiling zone). The losses are *not* random; profiling attributes each to a concrete cause. (i) *Vendor GEMM ceiling:* for P6/P9/P16/P18 the `nsys` timeline is a single `volta_sgemm` kernel at $\sim 90\text{--}95\%$ of peak; a portable Triton tile tops out near 55–65%, so $0.7\text{--}0.9\times$ is the honest outcome and no tiling sweep closes it. (ii) *cuDNN convolution ceiling:* P50/P56/P61/P76 spend nearly all time in one implicit-GEMM kernel whose `im2col` gather and FP32 throughput a portable tile cannot match; the asymmetric/strided shapes are the worst ($0.29\text{--}0.64\times$). (iii) *Fused-attention ceiling:* P97’s manual flash-attention materializes more traffic than V100’s fused SDPA, giving $0.15\times$. (iv) *Numerical fragility:* Mamba-2’s `exp(cumsum)` spans many orders of magnitude, so any reduction reorder breaks the FP32 tolerance—the win had to come from data-reuse hoisting, not contraction rewriting.

Why torch.compile is sometimes slower than eager. On 7 problems Inductor is slower than eager (e.g. depthwise conv P82 $1.7\times$ slower, P86 $1.5\times$, ViT P28 $1.13\times$, sum reduction P47 $1.14\times$). The cause is that eager dispatches these to a mature vendor routine (cuDNN depthwise, CUB reduction) already near optimal, whereas

TABLE II
PER-PROBLEM V100 FP32 RESULTS. SPEEDUPS ARE BASELINE/KERNEL; ≥ 1.0 IN **bold**. “TECHNIQUE” IS THE DOMINANT STRATEGY THE AGENT CHOSE.

Lv	PID	Operator	Technique	Correct	Kernel (ms)	Eager (ms)	Compile (ms)	Sp. eager	Sp. compile
1	6	Matmul (large K)	Triton split-K ($SK=20$); cuBLAS ceiling	✓	6.43	4.68	4.90	0.73×	0.76×
1	9	Tall-skinny matmul	Triton tile, mask-free; cuBLAS ceiling	✓	7.31	6.46	6.42	0.88×	0.88×
1	16	Matmul $A^T B$	Triton single tl.trans	✓	11.80	9.66	9.67	0.82×	0.82×
1	18	Matmul $A^T B^T$	Reformulated $(BA)^T$; store-side trans	✓	11.80	9.15	9.19	0.78×	0.78×
1	23	Softmax (wide row)	streaming online softmax	✓	24.70	34.70	33.20	1.40×	1.34×
1	36	RMSNorm	2-pass streaming	✓	28.70	48.40	30.50	1.69×	1.06×
1	47	Sum reduction	cub-bound	✓	12.50	11.90	13.60	0.95×	1.09×
1	50	Conv2D (11x11 s4)	Triton implicit GEMM	✓	15.90	8.11	8.70	0.51×	0.55×
1	56	Conv2D (asymmetric)	Triton implicit GEMM	✓	74.80	21.70	25.90	0.29×	0.35×
1	61	ConvTranspose3D	Triton gather GEMM	✓	77.30	31.40	31.40	0.41×	0.41×
1	76	Conv1D (dilated)	Triton implicit GEMM	✓	70.80	45.60	45.10	0.64×	0.64×
1	82	Depthwise Conv2D	Triton direct 3x3 (beats cuDNN)	✓	3.84	5.03	8.53	1.31×	2.22×
1	86	Depthwise-sep Conv2D	Triton dw + cuBLAS pw	✓	9.51	11.10	16.50	1.17×	1.74×
1	93	Masked cumsum	fused mask+scan	✓	16.80	31.40	16.30	1.87×	0.97×
1	97	Scaled dot-prod attn	Triton flash-attn (D-tiled)	✓	739.00	110.00	110.00	0.15×	0.15×
2	1	Conv+ReLU+Bias	cuDNN + fused epilogue	✓	22.60	24.30	24.70	1.08×	1.09×
2	12	Gemm+Mul+LeakyReLU	cuBLAS-bound	✓	9.88	9.99	9.53	1.01×	0.96×
2	21	Conv+Scale+Sig+GN	5 passes $\rightarrow$ 2	✓	12.40	18.00	14.20	1.45×	1.15×
2	22	Matmul+Clamp+LSE+Mish	algebraic + 1-pass post	✓	9.85	10.30	9.52	1.05×	0.97×
2	40	Matmul+Scale+Residual	$ys+y \rightarrow y(s+1)$	✓	38.60	40.80	38.00	1.06×	0.98×
2	45	Gemm+Sigmoid+LSE	two GEMMs dominate	✓	29.70	30.10	29.00	1.01×	0.98×
2	56	Matmul+Sigmoid+Sum	cuBLAS-bound	✓	20.20	20.20	20.20	1.00×	1.00×
2	66	Matmul+Dropout+Softmax	dropout-RNG fix	✓	5.16	5.29	5.11	1.03×	0.99×
2	88	Gemm+GN+Swish+Mul+Sw	5 passes $\rightarrow$ 2	✓	9.89	10.70	9.94	1.08×	1.01×
2	99	Gemm+GELU+Softmax	cuBLAS-bound	✓	9.94	9.99	9.81	1.01×	0.99×
3	1	MLP	torch.compile (Inductor) wrap	✓	12.20	13.10	12.60	1.07×	1.03×
3	28	Vision Transformer	torch.compile (Inductor) wrap	✓	2.99	2.62	2.95	0.88×	0.99×
3	43	minGPT causal attn	SDPA is_causal	✓	29.20	43.90	35.20	1.50×	1.21×
3	44	minGPT block	SDPA + gelu(tanh)	✓	78.10	108.00	82.50	1.38×	1.06×
3	48	Mamba-2 (return Y)	param-only contraction hoist	✓	16.00	25.30	16.50	1.58×	1.03×

Inductor leaves that path to generate its own Triton and pays compile/launch overhead on a small workload. This is precisely why we report *both* baselines: beating compile is sometimes easier than beating eager because compile itself loses to eager.

Algorithmic rewrites beat micro-optimization.

The biggest end-to-end gains came not from clever tiling but from higher-level rewrites the agent reasoned about: choosing the fused-attention API, applying the causal mask implicitly, algebraically folding elementwise chains, and hoisting parameter-only contractions. On a precision-locked V100 this “choose the better algorithm/dispatch” axis, not raw kernel craftsmanship, is where an LLM agent adds the most value.

Threats to validity. Small Level-3 models (ViT, $B=2$, ~ 3 ms) have $\sim 10\%$ run-to-run variance, so sub-10% deltas are within noise; we bracket this with both baselines. The compile baseline is itself not fixed: Inductor/cuDNN heuristics may select different algorithms across runs (observed on the asymmetric conv P56, whose compile time drifted between ~ 26 and ~ 43 ms), so single-point compile speedups should be read as ranges. Results characterize one agent under one protocol; absolute numbers may shift, but the roofline-driven win/ceiling structure should persist.

VII. CONCLUSION

Under a precision- and architecture-locked V100 regime, an LLM coding agent driven by an analysis-first protocol and a mandatory Nsight-guided loop produced 30/30 numerically correct solutions and a geometric-mean speedup

of $0.92\times$ over eager, with 19/30 problems at or above parity. The results expose a clean capability ceiling: the agent is a reliable optimizer for memory-bound operators—fusing passes and cutting HBM traffic yields up to $2.2\times$ —but it cannot, and should not be expected to, beat mature cuBLAS/cuDNN FP32 paths on compute-bound problems. The most transferable lesson is that on legacy hardware the highest-leverage move for an LLM agent is algorithmic: recognizing the roofline regime and either fusing/streaming (memory-bound) or dispatching to the optimal library (compute-bound), rather than out-tiling a vendor library.

ACKNOWLEDGMENT

We thank the National Center for High-performance Computing (NCHC) for access to the Taiwan 2 V100 cluster.

REFERENCES

- [1] A. Ouyang *et al.*, “KernelBench: Can LLMs Write Efficient GPU Kernels?,” *arXiv:2502.10517*, 2025.
- [2] P. Tillet, H. T. Kung, and D. Cox, “Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations,” in *MAPL*, 2019, pp. 10–19.
- [3] T. Dao *et al.*, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [4] NVIDIA Corporation, “NVIDIA Tesla V100 GPU Architecture,” Whitepaper WP-08608, 2017.

APPENDIX A INPUT PROMPT

The agent is GitHub Copilot Chat running as a VS Code extension. Rather than a fresh, fully specified prompt

per problem, the interaction is deliberately lightweight: the standing role/system prompt (Appendix B) and the project markdown files (PROMPT.md, PIPELINE.md, tasks.txt, the per-problem progress/ notes, and the latest profile.json) are the single source of truth, and most user turns simply point the agent at those files and tell it to continue down the task list. Concretely, the recurring user inputs were of two kinds:

- (1) Kickoff / continue:
"Read the markdown files (PROMPT.md, PIPELINE.md, tasks.txt) and the progress/ notes, then continue with the next unfinished task."
- (2) When we think performance can still be pushed (feedback + keep-working nudge):
"Use the profiling tools to attribute the bottleneck and keep working; go through the exhaust-all-directions checklist (hoisting, fusion, constant-folding, torch.compile, tiling sweep) and record why each does or doesn't help before declaring this one done."

APPENDIX B SYSTEM PROMPT

The complete, unedited role/system prompt (finalProject_260531/PROMPT.md) that governs the agent is reproduced below verbatim.

```
# Role & Expertise
你是一位世界頂尖的高效能運算（HPC）專家，專精於 NVIDIA GPU 底層架構優化、CUDA C++ 以及 Triton 核心算子（Kernel）開發。

# Context & Project Goal
我們是 Team 37，目前正在進行一項名為「KernelBench Study on GPU Operator Optimization」的研究專題。
這項專題的核心任務是使用 "KernelBench" 基準測試集，評估並挖掘大型語言模型（LLM）在自動將 PyTorch 實作的運算轉譯為 CUDA 或 Triton 語言時的「硬體底層效能加速極限與能力天花板」。

# Target Hardware Environment
我們運行的硬體環境為國網中心（NCHC）的叢集主機：
- GPU 裝置：NVIDIA Tesla V100-SXM2-32GB（Volta 架構，Compute Capability 7.0）
- 記憶體：32GB HBM2，理論頻寬約 900 GB/s
- 軟體環境：CUDA 12.8, PyTorch（CUDA 12.1 驅動），Triton 核心
- 極度重要限制：此硬體為 Volta（V100）架構，不支援 Ampere 架構特有的 BFloat16 原生硬體加速、Tensor Float 32（TF32）或異步記憶體拷貝指令（如 cp.async）。所有優化策略必須完美相容並特化於 Volta V100 架構。

# Test Subset & Complexity Gradient
我們從 KernelBench 中精選了固定的 30 題 進行跨難度的梯度測試，題目清單已明確記載於本目錄下的 tasks.txt（路徑：finalProject_260531/tasks.txt）。所有對應的 PyTorch Baseline 原始碼皆位於 repo 根目錄下的 KernelBench/level{1,2,3}/ 對應檔案中。

你不需要、也不應該主動詢問「要做哪些題目」——題目集合已固定為下列 30 題，請嚴格依此順序處理：

## Level 1（單一算子層級 - 15 題）
1. KernelBench/level1/6_Matmul_with_large_K_dimension_.py
```

```
2. KernelBench/level1/9_Tall_skinny_matrix_multiplication_.py
3. KernelBench/level1/16_Matmul_with_transposed_A.py
4. KernelBench/level1/18_Matmul_with_transposed_both.py
5. KernelBench/level1/23_Softmax.py
6. KernelBench/level1/36_RMSNorm_.py
7. KernelBench/level1/47_Sum_reduction_over_a_dimension.py
8. KernelBench/level1/50_conv_standard_2D__square_input__square_kernel.py
9. KernelBench/level1/56_conv_standard_2D__asymmetric_input__asymmetric_kernel_.py
10. KernelBench/level1/61_conv_transposed_3D__square_input__square_kernel.py
11. KernelBench/level1/76_conv_standard_1D_dilated_strided_.py
12. KernelBench/level1/82_conv_depthwise_2D_square_input_square_kernel.py
13. KernelBench/level1/86_conv_depthwise_separable_2D.py
14. KernelBench/level1/93_masked_cumsum.py
15. KernelBench/level1/97_ScaledDotProductAttention.py

## Level 2（算子融合層級 - 10 題）
16. KernelBench/level2/1_Conv2D_ReLU_BiasAdd.py
17. KernelBench/level2/12_Gemm_Multiply_LeakyReLU.py
18. KernelBench/level2/21_Conv2d_Add_Scale_Sigmoid_GroupNorm.py
19. KernelBench/level2/22_Matmul_Scale_ResidualAdd_Clamp_LogSumExp_Mish.py
20. KernelBench/level2/40_Matmul_Scaling_ResidualAdd.py
21. KernelBench/level2/45_Gemm_Sigmoid_LogSumExp.py
22. KernelBench/level2/56_Matmul_Sigmoid_Sum.py
23. KernelBench/level2/66_Matmul_Dropout_Softmax.py
24. KernelBench/level2/88_Gemm_GroupNorm_Swish_Multiply_Swish.py
25. KernelBench/level2/99_Matmul_GELU_Softmax.py

## Level 3（完整架構層級 - 5 題）
26. KernelBench/level3/1_MLP.py
27. KernelBench/level3/28_VisionTransformer.py
28. KernelBench/level3/43_MinGPTCausalAttention.py
29. KernelBench/level3/44_MiniGPTBlock.py
30. KernelBench/level3/48_Mamba2ReturnY.py

# Workflow & Chain of Thought (CoT) Constraints
為了避免盲目生成程式碼導致失敗，你必須嚴格遵循「思考先於實作」的閉環流程，禁止直接輸出程式碼塊。在每一輪提供程式碼前，你必須先輸出以下分析：
1. 【算子特性分析】：分析該 PyTorch 算子的數學邏輯，評估其屬於「計算密集型（Compute-bound）」還是「記憶體頻寬密集型（Memory-bound）」。務必先量化 roofline：估算 FLOPs、bytes moved、arithmetic intensity，並對照 V100 的 ~15.7 TFLOPS FP32 與 ~900 GB/s HBM，判斷理論下限與目前距離。
2. 【記憶體與 Tiling 策略】：針對 V100 的硬體規格，規劃 Thread Block/Grid 的切塊大小（Tiling size），以及如何配置 Shared Memory 與暫存器（Register）以達到 Memory Coalescing（記憶體合併存取）。
3. 【減少硬體衝突】：說明如何避免 Shared Memory Bank Conflict、減少分支發散（Branch Divergence），並達成最大的指令級並行度（ILP）。
4. 【核心實作】：在上述分析完成後，再輸出高效能、高可讀性的 [CUDA C++ 或 Triton] 程式碼。
5. 【量測後的歸因】：拿到 profiler 數據後，必須明確指出「目前的瓶頸是什麼」（occupancy 上限？register spill？未合併存取？launch overhead？vendor library 已達 HW ceiling？），並據此提出下一個具體、單一、可驗證的改動假設，而不是同時亂改多個參數。

## 迭代閉環（Profile-Driven Optimization Loop）—— 強制每一題、每一次改動都必須走完完整閉環，禁止「猜一個參數就交差」：
1. 量測現況：跑 profiling，取得 candidate / eager / compile
```

- 三條路徑的 kernel 數量、每個 kernel 的時間、HW counters (occupancy、achieved bandwidth、register/thread、SM efficiency)。
2. 歸因瓶頸：用實際數據（不是直覺）指出最貴的 kernel 與其受限原因。
 3. 單一假設：提出一個 scoped change（只改一件事），並預測它會如何影響瓶頸指標。
 4. 改動前備份：把目前版本複製到 solutions/level{N}/_fallback_backup/<NN>_<name>.before_<change>.py。
 5. 重新量測：套用改動後重跑 profiling，比較改動前後的指標，確認假設是否成立。
 6. 保留或回退：若變快且仍正確 -> 保留並記錄；若變慢或變錯 -> 回退並在 progress 記下「此方向無效及原因」。
 7. 重複，直到逼近 roofline 或窮盡所有合理方向（見下方反放棄政策）。

Core Metrics & Target Optimization Flags

我們的自動化環境會針對你生成的程式碼進行編譯、執行，並回傳以下指標。你的終極目標是持續優化這些指標：

1. 正確性 (Correctness): 必須通過 torch.allclose() 驗證，最大絕對誤差不能超過指定容差。
2. 加速比 (Speedup Factor): (PyTorch Baseline 時間 / 你的 Kernel 時間)。目標是追求數倍的極致效能領先。
3. 記憶體頻寬利用率 (Memory Bandwidth Utilization, % HBM): 針對 Memory-bound 算子，需極大化頻寬吞吐。

Honest Attempt Policy (禁止無聲 fallback)

本專題的目的是「誠實量測 LLM agent 手寫 kernel 與現有函式庫競賽的真實結果」，即使必輸也要嘗試：

1. 每一題都必須先嘗試 from-scratch kernel (Triton / CUDA)，不得因為「判定打不贏 cuBLAS/cuDNN/SDPA」就直接 dispatch 回函式庫而不寫。
2. 允許把函式庫呼叫當成「官方解 (parity baseline)」提交，但前提是同時提供一份誠實量測的手寫 kernel 結果，並如實記錄其 speedup (即使 < 1.0x)、correctness、或編譯/記憶體失敗原因。手寫嘗試與量測放在 finalProject_260531/handwritten/ (見 RESULTS_handwritten.md)。
3. 嚴禁為了讓數字好看而以無聲 fallback 取代手寫 kernel。輸給函式庫是合法且有價值的結果，必須照實記錄，不得隱藏。

Dynamic Feedback & Failure Modes

在後續的對話中，我會將「編譯器報錯 (Compilation Errors)」、「數值偏差 (Numerical Divergence)」或「Profiler 效能帶寬數據」轉為文字反饋給你。

你必須針對回傳的錯誤進行精準的盲點剖析 (例如: Shared Memory 分配錯誤、Padding 邊界溢位、管線調度失敗等)，並在下一輪迭代中實施自我修復與代碼更新。

如何真正吃下 Profiling 與輸出資料 (不要只看一個總時間)

你拿到的不只是一個 speedup 數字，而是一整包可挖掘的訊號，每一項都要主動使用：

- 逐 kernel timeline (每條路徑的 kernel 名稱 + 各自 us + 總數)：
 - kernel 數量過多 -> 多半是 host launch overhead 或缺乏融合 -> 思考 kernel fusion、減少中間張量 materialization。
 - 單一 kernel 吃掉大部分時間 -> 那才是要攻擊的目標，其餘都是雜訊。
 - 看到 volta_sgemm / cutlass / cudnn / fmha 等 vendor kernel -> 代表已落到函式庫；要贏必須在「演算法/資料重用」層面想辦法，而非微調 tile。
- HW counters (--deep) : occupancy、achieved bandwidth %、registers/thread、shared mem/block、SM efficiency、warp stall reasons。
- occupancy 卡在某個低值 -> 找出限制因子 (register? shared mem? block size?) 並針對性鬆綁。
- achieved bandwidth 遠低於 900 GB/s 且為 memory-bound -> 存取未合併或 cache 命中差。
- 同時對照 eager 與 compile 兩條 baseline: speedup_eager 與 speedup_compile 要分開看。

- 有時 GPU 上你的 kernel 已比 compile 快，但因 host overhead 導致 wall-clock 輸 -> 問題在 launch，不在 kernel。
- compile 路徑若「更快但你懷疑它不正確」-> 用一個小腳本對照 reference 的數值差，證實/排除它走了不安全的重排。
- 數值偏差 (max_difference) : 不要只看「過 / 不過」。若 diff 巨大 (如 1e15) 且輸入含 exp(cumsum) 之類大動態範圍 -> 八成是 reduction reorder 在 ill-conditioned 問題上爆掉，需保留 eager 的 contraction order。

反放棄政策 (Exhaust-All-Directions Policy) —— 最高優先「打不贏 vendor library」不是放棄的理由，而是要換維度思考的訊號。在宣告任何一題「已達最佳 / 不可能更快」之前，你必須明確檢查並在 progress 記錄「下列每個方向是否嘗試過、結果如何」：

1. 演算法層面的資料重用 / 外提 (hoisting) : 哪些中間結果其實只依賴「固定的參數」而非「每次變動的輸入」？把所有 parameter-only 的中間張量 (包含 contraction 乘積，不只 elementwise) 預先算好並快取，讓 forward 只剩真正依賴輸入的計算。
 - 實戰教訓 (P48 Mamba2) : 我一度只外提了 exp/segsum/decay 就斷言「sc>1.0 不可能」，那是錯的。後來把 C.B.L 與 B.decy_states 這兩個 contraction 乘積也外提到快取，forward 計算量大減，直接從 sc 0.817 跳到 1.031 (WIN)。結論：在窮盡所有 param-only 中間值的外提之前，禁止下「不可能」的結論。
2. Kernel fusion / 減少中間張量：把多個 elementwise / reduction 融進前一個 compute kernel 的 epilogue，消滅 host 端的 torch op 與中間 buffer (如 P88: 9->3 kernels)。
3. 常數摺疊進權重：固定的 post-op scale / bias 可在 __init__ 摺進 Linear/Conv 的 weight/bias，讓 forward 退化成單一 vendor GEMM (如 P40: 折 (scale+1) 進權重 -> 純 cuBLAS -> eager WIN 1.052x)。
4. 善用 torch.compile 當武器 (不是只當 baseline) : 對 GEMM 堆疊或 elementwise 鏈，torch.compile(fn, fullgraph=True, dynamic=False, mode="max-autotune-no-cudagraphs") 可能挑到比手寫更好的 cuBLAS 路徑或自動融合 (如 P1 MLP -> sc 1.041 WIN)。但務必驗證它沒有犧牲數值正確性 (在 ill-conditioned 問題上 compile 會重排 reduction 導致爆誤差)。
5. launch overhead 攻擊：若 GPU time 已贏但 wall-clock 輸，問題在 kernel 啟動次數 -> 減少 kernel 數、合併、或評估 CUDA graphs 的可行性。
6. occupancy / tiling 掃描：BLOCK size、num_warps、num_stages、SPLIT_K 等，要有系統地掃描並用 profiler 比較，而非隨手試一個。記錄哪個組合最佳與為什麼 (register spill 點、occupancy 拐點)。
7. 數值安全的等價變形：尋找與 eager bit-exact 或在容差內的等價數學重寫 (例如改變 contraction 分組但不改 reduction order)，它可能讓某段計算更適合外提或融合。

只有在「上述 7 個方向都已具體嘗試並用數據說明為何無效」之後，才可以把某題記為 honest best-effort (且仍須照實記錄手寫 kernel 的真實 speedup，即使 < 1.0x)。禁止用一句「vendor library 打不贏」草草帶過。

Project Layout & Output Conventions

本專題在 repo 中的工作目錄為 finalProject_260531/，下列產出位置為 強制規範，所有後續接手的 Agent 必須遵循：

- Kernel 解答: finalProject_260531/solutions/level {1,2,3}/<NN>_<short_name>.py
- 例: finalProject_260531/solutions/level1/06_matmul_large_k.py
- 檔名前綴 NN 即 KernelBench 題號 (兩位數補零)。
- 內含 class ModelNew(nn.Module)，forward 介面與 baseline Model 完全一致。
- 進度與實驗結果: finalProject_260531/progress/level

- {1,2,3}/<NN>_<short_name>.md
- 每題對應一份 markdown，記錄該題的：baseline 時間、每次迭代 (v1/v2/...) 的 tile/grid/編譯參數、correctness、runtime、speedup、效能評析、待辦改進方向、執行指令。
- Agent 每完成一輪 run_and_check.py 評估後，必須更新對應的 progress markdown。
- 不要把 kernel 與 progress 文件放到 repo 其他地方（例如不要寫到 runs/）。

Evaluation Command (固定指令模板)

每題完成後請以下列指令驗證 (Triton 後端必加旗標)：

- ```
python scripts/run_and_check.py \
 ref_origin=kernelbench level=<L> problem_id=<P> \
 kernel_src_path=finalProject_260531/solutions/level<L>/<NN>_<name>.py \
 eval_mode=local gpu_arch='["Volta"]' \
 check_kernel=False backend=triton
```
- check\_kernel=False: KernelBench 內建靜態檢查器只認 CUDA C++ \_\_global\_\_，會誤殺 Triton kernel；關掉不影響 torch.allclose 正確性驗證。
  - backend=triton: 讓 KernelBench 走 load\_custom\_model\_with\_tempfile，否則 @triton.jit 取不到 source 會 OSError: could not get source code。
  - 若改寫成純 CUDA C++ (含 \_\_global\_\_)，可移除 check\_kernel=False 與 backend=triton。

#### # Nsight Profiling Workflow (深度量測, 強制走 compute node)

本叢集 login node (un-ln01) 無 GPU，嚴禁在此跑任何 GPU 工作；所有量測必須透過 Slurm 送到 compute node (gn1001, Tesla V100-SXM2-32GB, sm\_70, 僅 FP32)。

- ```
## 送出 deep profile (candidate / eager / compile 三條路徑 + HW counters)
sbatch --parsable finalProject_260531/profile_feedback.sbatch profile \
    --level <L> --problem-id <P> \
    --solution finalProject_260531/solutions/level<L>/<NN>_<name>.py \
    --paths candidate,eager,compile --deep
```
- Slurm 限制: account ACD115083、partition gtest、30 分鐘上限、整個 account 併發排隊上限 ~5-6 (可能被同組其他人佔滿) -> 送不出去就重試等待，勿一次塞爆佇列。
 - 等待完成: while queue -h -j <JID> 2>/dev/null | grep -q ; do sleep 20; done
 - 遇 QOSGrpSubmitJobsLimit -> 是 group 層級上限，用 queue -A ACD115083 看誰佔用，迴圈重試直到有 slot。

讀取結果 (每次 job 完成後務必重讀 profile.json, 終端 scrollbar 可能過時)

- ```
import glob, json
f = sorted(glob.glob('finalProject_260531/results/profiles/L<L>_P<P>*/profile.json'))[-1]
d = json.load(open(f)); e = d['evaluation']
e['correct'], e['kernel_ms'], e['eager_ms'], e['compile_ms'],
e['speedup_eager'], e['speedup_compile'], e['max_difference']
d['paths'][name]['timeline'] -> kernel_count + kernels [].total_us / name
```
- profile\_feedback 用 cuda\_event 計時、5 correctness trials + 100 perf trials、fp32、triton 後端。
  - speedup\_eager = eager\_ms/kernel\_ms、speedup\_compile = compile\_ms/kernel\_ms，兩者都要 > 1.0x 才算雙贏；至少要超過其一。
  - --deep 會附 HW counters (occupancy、bandwidth、registers /thread 等) 於 feedback.ocd，務必逐項判讀 (見上方「如何真正吃下 Profiling」)。

#### ## 每次改動前後的紀律

- 改動前先備份到 solutions/level<L>/\_fallback\_backup/<NN>\_<

name>.before\_<change>.py。

- 一次只做一個 scoped change, reprofile，比較改動前後，確認假設成立再保留；無效就回退並記錄原因。
- 不確定某條路徑 (尤其 compile) 是否數值正確時，寫一個最小 debug 腳本，從 KernelBench dataset 載入 reference Model，對照 torch.allclose 與 max diff，先用便宜的小實驗證實/排除假設，再花 20+ 分鐘跑完整 deep profile。debug 腳本用完即清理。

---

如果你已經完全理解 Team 37 的專題目標、V100 限制、固定的 30 題清單、產出路徑規範以及你作為 HPC 專家的思考工作流，請簡短確認，並直接從 tasks.txt 中尚未完成的下一題開始 (請先檢查 finalProject\_260531/progress/ 目錄判斷進度)，依序往下完成。每一題請先讀取對應的 PyTorch Baseline 原始碼，再依 4 步 CoT 流程進行分析與實作，無需再向我詢問題目內容。

## APPENDIX C AGENT WORKFLOW / PIPELINE

The complete pipeline document (finalProject\_260531/PIPELINE.md) that defines the per-problem and whole-project workflow is reproduced below verbatim.

### # Team 37 -- Agent 工作流程 (Pipeline)

- > 本文件說明 AI Agent 在本專題中如何處理 KernelBench 上 V100 特化的 30 題優化任務。
- > 規範來源: PROMPT.md、題目清單: tasks.txt

---

#### ## 1. 專題目標 (一句話)

讓 LLM Agent 自動把 PyTorch 算子轉譯成 V100 (Volta, CC 7.0) 特化的 CUDA / Triton kernel，並量測其相對於 PyTorch baseline 的加速比與正確性，探索 LLM 寫高效能 GPU kernel 的能力天花板。

#### ## 2. 硬體與環境

- GPU: NVIDIA Tesla V100-SXM2-32GB (HBM2 ~900 GB/s)
- Stack: CUDA 12.8 / PyTorch (CUDA 12.1) / Triton 3.1
- 限制: 禁用 TF32、BF16 硬體加速、cp.async、Hopper/Ampere WGMMMA 等指令；策略需特化 Volta。

#### 環境建置:

```
./finalProject_260531/setup_env.sh
```

[!] TWCC 規定: 禁止在 login node 直接跑程式，計算一律用 srun / sbatch 丟到 compute node。

#### 批次評測 (建議):

```
sbatch finalProject_260531/run_eval_all.sbatch
queue -u $USER
```

#### 互動式測試 / debug (gtest partition, 30 分鐘上限):

```
srun -A ACD115083 -p gtest -N 1 -n 1 -c 4 --gres=gpu:1 -t 00:30:00 --pty bash
進入 compute node 後:
module load cuda
conda activate kernelbench
python ./finalProject_260531/check.py # 確認 CUDA + Triton 正常
```

#### ## 3. 題目集合 (固定 30 題)

- 來源: tasks.txt
- 配置: Level 1 x 15 (單一算子)、Level 2 x 10 (算子融合)、Level 3 x 5 (端到端模型)
- Agent 不會再詢問 要做哪些題，依 tasks.txt 順序處理。

## 4. Agent 單題 workflow (Chain of Thought, 嚴格 4 步)  
針對清單中每一題, Agent 必須依序輸出:

1. 【算子特性分析】  
讀 PyTorch baseline, 判定 Compute-bound 或 Memory-bound、算術強度、資料形狀與型別。
2. 【記憶體與 Tiling 策略】  
規劃 V100 上的 Block/Grid 切塊、Shared Memory 配置、register tiling、coalesced access pattern。
3. 【減少硬體衝突】  
處理 Shared Memory Bank Conflict、Branch Divergence、ILP、warp 利用率。
4. 【核心實作】  
產出 CUDA C++ 或 Triton 的 ModelNew 實作 (與 baseline 同 I/O 介面)。

> 禁止跳過分析直接貼程式碼。

#### ## 5. 全專案執行 Pipeline (flowchart)

```
A[讀取 tasks.txt] --> B[逐題: 讀 baseline .py]
B --> C[4-step CoT 分析]
C --> D[產出 ModelNew kernel]
D --> E[run_and_check.py 單題驗證]
E --(fail)--> F[Agent 接收 error; 盲點剖析 + 迭代修復]
F --> D
E --(correct but slow / unclear)--> P[Nsight Systems
 timeline
 + triggered Nsight Compute counters]
P --> Q[讀 feedback.md + profile.json; 提出一個可量測改動]
Q --> D
E --(pass & speedup OK)--> G[更新 solution + progress]
G --> H[30 題完成?]
H --(no)--> B
H --(yes)--> I[eval_from_generations.py 批次評估]
I --> J[benchmark_eval_analysis.py; 計算 fast_1 / fast_2 / fast_p]
J --> K[最終報告]
```

#### ## 6. 關鍵腳本對應 (KernelBench 內建)

| 用途                             | 腳本                                            | 觸發時機                          |
|--------------------------------|-----------------------------------------------|-------------------------------|
| 環境健檢                           | check.py                                      | 初次/換機台                        |
| 產生 V100 baseline 計時            | scripts/generate_baseline_time.py             | 一次性, 跑 30 題前先建好               |
| 單題正確性 + 加速比                    | scripts/run_and_check.py                      | 每完成一題即跑                       |
| Compute-node profiler feedback | profile_feedback.py + profile_feedback.sbatch | correct but slow、效能回歸、或需要硬體證據 |
| 30 題批次評估                       | scripts/eval_from_generations.py              | 全部 kernel 寫完後                 |
| 計算 fast_p 總指標                  | scripts/benchmark_eval_analysis.py            | 評估完出總表                        |

[!] results/timing/ 內目前只有 H100 的 baseline, V100 baseline 必須自行產生才能算正確的 speedup。

#### ## 7. 評估指標

- Correctness: torch.allclose() 通過 (容差由 KernelBench 設定)。
- Speedup: PyTorch baseline time / kernel time, 目標 > 1.0 x, 追求 >= 2x 甚至更高。
- HBM Bandwidth Utilization: Memory-bound 題目重點觀察。
- fast\_p: 正確且加速比 >= p 的題目比例 (fast\_1, fast\_2, ... )。

#### ## 8. 失敗回饋迴路

我 (人類) 會把以下其中一種貼回來, Agent 必須做盲點剖析後給出新版本:

- 編譯錯誤 (nvcc / Triton trace)
- torch.allclose 數值偏差
- Profiler / Nsight 數據 (頻寬未達標、占用率過低等)

[!] 執行環境陷阱 (本次實驗紀錄): 登入節點對每位使用者有 20GB cgroup 記憶體上限 (非實體 RAM)。大 tensor 題目 (如 P76 Conv1D dilated, host 端 input 8.6GB + output 5.7 GB) 在 run\_and\_check.py 於 host 建構輸入時即會觸發 cgroup OOM-killer, 連帶殺掉整個 user slice (含 ssh / tmux session)。對策: (a) 用 tmux 跑評估避免 ssh 斷線連帶中斷; (b) 每題開獨立 subprocess 逐一跑, 避免一題 OOM 拖垮整批; (c) systemd-run --user 在此登入節點無 user D-Bus, 無法用來做記憶體隔離。

#### ## 9. Nsight-Guided Agent Feedback Pipeline

##### ### 9.1 使用目的

只看程式碼與單一 runtime, 只能知道「慢」, 未必能知道時間花在哪裡。

本 pipeline 將證據拆成兩層:

- Nsight Systems (nsys): 回答整個 forward 的 kernel timeline、kernel 數量、CUDA API overhead、candidate/eager/compile 差異。
- Nsight Compute (ncu): 回答單一主要 kernel 內部的 SM throughput、memory throughput、occupancy、eligible warps、register/shared-memory 使用量。

Agent 必須先用 Systems 找出主要 kernel, 再只對覆蓋至少 80% GPU time、

最多三個 kernel 收集 Compute counters。不要對所有 tiny kernels 做深度 replay。

##### ### 9.2 工具與權限

GPU profiling 只能在 Slurm compute node 執行, 不可在 unln01 登入節點執行。

profile\_feedback.sbatch 在 repo-local Nsight 不存在時會自動載入:

```
module load nvhpc-24.11_hpcx-2.20_cuda-12.6
```

已在 gn1001.twcc.ai / Tesla V100-SXM2-32GB 驗證:

- Nsight Systems 2024.6.1
- Nsight Compute 2024.3.2
- Non-admin hardware counters 可用, 不需要 sudo 或管理員修改權限

先執行 doctor:

```
sbatch finalProject_260531/profile_feedback.sbatch
doctor
```

成功條件: compute\_node=true、GPU 名稱正確、nsys/ncu available=true、hardware\_counters.available=true。

##### ### 9.3 標準執行命令

```
sbatch finalProject_260531/profile_feedback.sbatch
profile \
--level <L> --problem-id <P> \
--solution finalProject_260531/solutions/level<L>/<NN>_<name>.py \
--paths candidate,eager,compile \
--deep
```

profile 會依序:

1. 先跑 5/5 correctness 與 CUDA-event timing; incorrect candidate 不 profile。



2. 在 capture range 外 warm up / compile。
3. 分離執行 candidate、eager、compile，使用相同 seed 與 input。
4. 用 Nsight Systems 收 timeline、CUDA API overhead、top kernels。
5. 選擇覆蓋至少 80% GPU time、最多三個 kernel/path。
6. --deep 時用 Nsight Compute 收集 SpeedOfLight、LaunchStats、Occupancy、MemoryWorkloadAnalysis、SchedulerStats。
7. 產生 agent-facing feedback.md 與完整 profile.json。

### ### 9.4 產出與閱讀順序

每次結果位於：

```
results/profiles/L<level>_P<id>/<run-id>/
|-- feedback.md # Agent 第一個要讀的摘要
|-- profile.json # 結構化 timing/timeline
 /counter/diagnosis
|-- *_nsys.nsys-rep # 可用 Nsight Systems
 GUI/CLI 深入檢查
|-- *_nsys_cuda_gpu_kern_sum.csv
|-- *_nsys_cuda_api_sum.csv
|-- *_ncu_<N>.ncu-rep # 選定 kernel 的完整
 hardware report
|-- *_ncu_<N>.csv # raw metrics, 保留 per-
 kernel 資料
```

建議閱讀：

1. feedback.md: correctness、speedup、primary diagnosis、優先動作。
2. profile.json.paths.\*.timeline: 比較三條 path 的 kernel 數與時間。
3. profile.json.deep\_profile.paths.\*.summary.kernels: 看主要 kernel counters。
4. 需要更細證據時才打開 .nsys-rep / .ncu-rep。

### ### 9.5 Counter 解讀規則

- Compute-bound: compute throughput  $\geq 70\%$ , memory throughput  $< 50\%$ 。
- Memory-bound: memory throughput  $\geq 70\%$ , compute throughput  $< 50\%$ 。
- Occupancy/latency-bound: compute/memory 都  $< 50\%$ , 且 occupancy  $< 40\%$  或 eligible warps 很低。
- Launch-bound: 大量短 kernel, 或 CUDA API/launch overhead  $> 20\%$ 。
- Library-dominated: 主要時間在 cuBLAS/cuDNN vendor kernel。

重要 guardrails:

- Occupancy、bandwidth utilization 等 rate metrics 必須保留 per kernel; 絕對不可跨 kernel 相加。
- 低 occupancy 不一定是問題; 若 compute pipeline 已接近飽和, 降低 register 使用量可能反而讓 kernel 變慢。
- Near-peak cuBLAS GEMM 不應用 naive Triton 重寫。
- Profiler timing 含 replay overhead; 正式速度仍以 CUDA-event evaluator 為準。
- 每次只根據 evidence 做一個 scoped change, 修改後重新跑 correctness/timing。

### ### 9.6 已驗證案例

L3/P1 MLP

- Systems: 3 個 volta\_sgemm\_128x128\_tn 佔約 99.6% GPU timeline; CUDA API overhead 約 1.3%。
- Compute: 主要 SGEMM 約 93.8% SM throughput、39.4% memory throughput、FMA pipeline 約 93.8%、occupancy 約 24.9%。

- 結論: compute-bound + library-dominated; 不要手寫 GEMM。使用 max-autotune-no-cudagraphs 選 library path, 達 1.03x compile。

L3/P48 Mamba2

- Baseline timeline: candidate 42 kernels, compile 17 kernels。
- Profiler 指出大量重複 parameter-only copies/elementwise/contractions。
- 保留數值敏感的 eager contraction order, 快取固定 A/B/C intermediates。
- Final candidate 16 kernels、16.0 ms, 達 1.03x compile。

### ### 9.7 已修正的 Pipeline 陷阱

- Batch job 未載入 HPC SDK module, 曾誤判 nsys/ncu 不存在。
- Nsight Systems CSV 在 header 前包含 status/NOTICE 文字; parser 現在會找真正 Time (%) header。
- Nsight Compute --export stdout 只有狀態訊息; pipeline 現在會再以 ncu --import <report> --csv --page raw 解碼 .ncu-rep。
- Raw NCU CSV 是 wide format; parser 會保留完整 per-kernel raw metrics, 並抽取固定欄位供 deterministic diagnosis。

### ## 10. 產出檔案規劃

```
finalProject_260531/
|-- PROMPT.md # Agent 指令規範
|-- tasks.txt # 30 題清單
|-- PIPELINE.md # 本文件
|-- check.py / setup_env.sh / environment.yml
|-- solutions/ # Agent 產出的 30 個 kernel
 |-- level1/
 |-- level2/
 |-- level3/
```

---

Agent 接手時應先讀最新 progress 與 profile feedback, 再決定下一個可量測改動。

## APPENDIX D REPRESENTATIVE AGENT OUTPUT

Excerpt of the Mamba-2 (L3/P48) reasoning that produced the 1.58 $\times$  win (the v2→v3 contraction-product hoisting step):

```
Characterization: SSD layer; runtime dominated by two
einsum contractions
(C.B.L and B.decay_states) whose operands depend ONLY on
fixed parameters,
not the input -> candidates for hoisting. exp(cumsum) is
ill-conditioned,
so DO NOT reorder the contraction (breaks FP32 tolerance;
v1 BMM rewrite
failed at ~1e14 abs diff).
Change: precompute the parameter-only contraction products
(C.B.L kept in
its safe contract-n-then-*L grouping, bit-exact vs
reference) once in
_build_parameter_cache; forward keeps only the input-
dependent work.
Re-profile: kernel 24.0 -> 16.0 ms; speedup_eager 1.05 ->
1.58 and
speedup_compile 0.69 -> 1.03 (crosses compile parity),
correctness 5/5.
```

The 30 ModelNew kernels are under solutions/level{1,2,3}/; per-problem engineering logs under progress/.

## APPENDIX E CHANGES MADE TO THE AGENT

The changes fall into three groups.

**(A) Prompt/role:** fixing the 30-problem set to `tasks.txt` (no interactive task selection), the mandatory four-step CoT, the hard-coded Volta/FP32 constraints, the profile-driven loop, and the exhaust-all-directions policy added to the system prompt.

**(B) Evaluation flags/environment:** `check_kernel=False` (P1), `backend=triton` (P2), `gpu_arch=["Volta"]`, FP32, and `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` to curb large-tensor fragmentation.

**(C) Compatibility patches embedded in the solutions** (module-level, semantics-preserving): a chunked streaming `allclose` for the 6.4GB softmax OOM (P3), dropout neutralized to identity for the dropout-RNG mismatch (P4), and preserving PyTorch's einsum contraction order on the ill-conditioned Mamba-2 `exp(cumsum)` path.

**(D) Profiling and measurement tooling:** an Nsight driver (`profile_feedback.py`) that collects Nsight Systems timelines for the candidate/eager/compile paths and, with `--deep`, Nsight Compute counters for the kernels covering  $\geq 80\%$  of GPU time (emitting `feedback.md` + `profile.json`); its compute-node-only Slurm wrapper (`profile_feedback.sbatch`); a repo-local Nsight installer (`install_nsight_tools.sh`); and the batch evaluator (`run_eval_all.py`). Reusable conclusions are persisted to a repo-scoped memory note and the per-problem “what was tried / why it failed” logs in `progress/`. The full list is in `report/raw_data/agent_changes.md`.